Tournament Management Platform

Master Plan, Part 3 of 4: Database Design (design level only, no model code)

Database: MongoDB Atlas + Mongoose, replica set so transactions work. Conventions: every document has *_id*, *createdAt*, *updatedAt*; mutable aggregate roots also have *version* (optimistic concurrency) and *schemaVersion*. References use ObjectId. Money-like values (purses, bids) are stored as **integers** (virtual credits), never floats. All times UTC.

1. Final collection list

#	Collection	Origin	Purpose (short)	First release
1	users	Blueprint	Accounts, roles, status, privacy settings	Essential
2	sessions	**New**	Server-side sessions for revocation	Essential
3	organizerApplications	Blueprint	Organizer approval workflow	Essential
4	accountRequests	Blueprint	Recovery and deletion requests (+ reset token hash)	Essential
5	tournaments	Blueprint	Tournament config, rules snapshot, status	Essential
6	tournamentRuleRevisions	**New**	History of rule edits and impact	Essential
7	invites	**New**	Invite-only registration codes	Essential
8	registrations	Blueprint	Player participation in a tournament	Essential
9	teams	Blueprint	Teams, captain, purse and squad counters	Essential
10	teamAssignments	Blueprint	Player-to-team assignment history (source of truth)	Essential
11	auctionSessions	**New**	Per-tournament auction config, state, round, countdown	Essential
12	auctionLots	Blueprint	One lot per auctioned player	Essential
13	auctionBids	Blueprint	Append-only bid and undo event log	Essential
14	fixtures	Blueprint	Matches, brackets, dependencies	Essential
15	matchResults	Blueprint	Current result state per fixture	Essential
16	resultRevisions	**New**	Immutable result/correction history	Essential
17	importJobs	**New**	CSV upload, parsed preview, validation	Essential (Phase S)
18	standingsSnapshots	Blueprint (optional)	Standings and qualification at points in time	Essential
19	announcements	Blueprint	Tournament announcements	Essential
20	notifications	Blueprint	Persistent in-site notifications	Essential
21	auditLogs	Blueprint	Append-only audit events	Essential
22	counters	**New**	Atomic sequences (waitlist position, etc.)	Essential

Not added: separate rounds collection (round is a counter on auctionSession and a field on bids), separate reports collection (reports are generated on demand), separate fixtureDependencies collection (dependencies are embedded on fixtures; see section 3), separate refresh-token collection (sessions covers it).

2. Collection detail

Format: principal fields, relationships, constraints and indexes. U = unique index; P = partial index. Fields are conceptual names.

users

Aspect	Design
Purpose	Identity, role and settings for every account.
Principal fields	name, emailNormalized, passwordHash, role (player | organizer | admin), organizerApproved (bool), status (active | suspended | deleted), mustChangePassword, settings {historyVisibility, ...}, deletedAt, anonymizedAt, lastLoginAt.
Relationships	Referenced by nearly everything (as actor, player, organizer, captain, recipient).
Indexes	U: emailNormalized. U partial: role=admin (guarantees at most one admin). Index: role+status.
Constraints / notes	Admin cannot be created through public API. On deletion: name and email replaced, credentials removed, document kept so ObjectIds in results/audit still resolve.

sessions

Aspect	Design
Purpose	Server-side session records enabling logout and revocation.
Principal fields	sessionIdHash, userId, createdAt, lastSeenAt, expiresAt, userAgent, ipHash, revokedAt.
Relationships	-> users.
Indexes	U: sessionIdHash. TTL index on expiresAt. Index: userId (revoke-all).
Constraints / notes	Raw session id exists only in the cookie; only its hash is stored.

organizerApplications

Aspect	Design
Purpose	Applications to become an organizer.
Principal fields	applicantId, details {organizationName, experience, contact, ...}, status (pending | approved | rejected), decidedBy, decisionReason, submittedAt, decidedAt.
Relationships	-> users (applicant, decider).
Indexes	U partial: applicantId where status=pending. Index: status+submittedAt.
Constraints / notes	Approval sets users.organizerApproved and writes an audit row in one transaction.

accountRequests

Aspect	Design
Purpose	Password recovery and account-deletion requests.
Principal fields	userId, type (recovery | deletion), status (open | verified | token_issued | completed | rejected), resetTokenHash, tokenExpiresAt, tokenUsedAt, reviewedBy, resolutionNote, history[] (small embedded status log), timestamps.
Relationships	-> users.
Indexes	Index: type+status+createdAt. U partial: userId+type where status in open states. TTL not used (history retained).
Constraints / notes	Token is random, stored hashed, single use, short-lived. Completion revokes all sessions.

tournaments

Aspect	Design
Purpose	Core tournament definition.
Principal fields	name, description, organizerId, format (knockout | league | group_knockout), status, visibility flags, registration {mode: public|invite, approval: auto|manual, opensAt, closesAt, maxParticipants, waitlistEnabled, reopenedCount}, dates {start, end}, teamCount, teamFormation (auction | manual), rules (points, tieBreakers, qualification, knockout rules), searchable text fields, cancelledReason, finalizedAt, version.
Relationships	-> users (organizer). Parent of nearly every tournament-scoped collection.

Aspect	Design
Indexes	Index: status+startDate, format+status, organizerId, text index on name/description. Index: registration.closesAt for deadline sweeps.
Constraints / notes	rules is embedded (bounded size); history lives in tournamentRuleRevisions. Status changes only through the lifecycle service. All tournaments stay publicly listable.

tournamentRuleRevisions

Aspect	Design
Purpose	Immutable record of each rule change and its impact preview.
Principal fields	tournamentId, changedBy, changedAt, reason, oldRules, newRules (diff), impactSummary {fixtures, standings, qualification}, recalculationStatus.
Relationships	-> tournaments, users.
Indexes	Index: tournamentId+changedAt.
Constraints / notes	Append-only. Large diffs store only changed paths.

invites

Aspect	Design
Purpose	Invite-only registration tokens.
Principal fields	tournamentId, codeHash, createdBy, boundUserId (optional), maxUses, usedCount, expiresAt, revokedAt.
Relationships	-> tournaments, users.
Indexes	U: codeHash. Index: tournamentId.
Constraints / notes	Redemption increments usedCount atomically (conditional update).

registrations

Aspect	Design
Purpose	A player's participation in one tournament.
Principal fields	tournamentId, playerId, status (pending | approved | rejected | waitlisted | withdrawn | removed), approvalMode, waitlistPosition, captainInterest {interested, note}, auctionEligible, withdrawnAt, removedReason, decidedBy, timestamps.
Relationships	-> tournaments, users (player).
Indexes	**U partial: (tournamentId, playerId) where status in [pending, approved, waitlisted]** (no duplicate active registration). Index: tournamentId+status. Index: playerId. Index: tournamentId+waitlistPosition.
Constraints / notes	Withdrawal and removal are status changes, never deletes. Capacity check + insert in one transaction against counters.

teams

Aspect	Design
Purpose	Teams within a tournament.
Principal fields	tournamentId, name, captainId (nullable), purseTotal, purseRemaining, squadCount, squadMax, status, version.
Relationships	-> tournaments, users (captain).
Indexes	U: (tournamentId, nameNormalized). U partial: (tournamentId, captainId) where captainId exists.
Constraints / notes	squadCount/purseRemaining are denormalized counters changed only inside the auction/assignment transaction; guarded by conditional updates ($inc with filter purseRemaining >= price and squadCount < squadMax).

teamAssignments

Aspect	Design
Purpose	Authoritative record of who belongs to which team, with history.
Principal fields	tournamentId, teamId, playerId, method (auction | manual), status (active | superseded | removed), price (if auction), lotId, assignedBy, assignedAt, previousAssignmentId, reason.

Aspect	Design
Relationships	-> tournaments, teams, users, auctionLots.
Indexes	**U partial: (tournamentId, playerId) where status=active** (a player on at most one team). Index: teamId+status.
Constraints / notes	Active rows define squads. Changing a squad appends a new row and marks the old superseded.

auctionSessions

Aspect	Design
Purpose	Auction configuration and live state for one tournament.
Principal fields	tournamentId, config {startingPurse, squadMax, basePrice, increment, countdownSeconds, orderMode, orderSeed, unsoldPolicy}, status (configured | live | paused | finished), currentLotId, roundNumber, countdownEndsAt, finalizedAt, version.
Relationships	-> tournaments, auctionLots.
Indexes	U: tournamentId. Index: status.
Constraints / notes	Config frozen at start. countdownEndsAt is an absolute UTC time the server persists and restores after restarts.

auctionLots

Aspect	Design
Purpose	One lot per eligible player.
Principal fields	tournamentId, auctionSessionId, playerId, orderIndex, status (pending | open | countdown | sold | unsold), basePrice, currentPrice, leaderTeamId, leaderBidId, roundNumber, finalPrice, winningTeamId, soldAt, version.
Relationships	-> auctionSessions, users, teams, auctionBids.
Indexes	U: (auctionSessionId, playerId). Index: auctionSessionId+status+orderIndex.
Constraints / notes	Bid acceptance is a single conditional update: filter {_id, status in [open,countdown], version, currentPrice}; sets new price, leader, version+1. Sold lot must carry finalPrice and winningTeamId (validated in service and tested).

auctionBids

Aspect	Design
Purpose	Append-only event log of bids and undo events.
Principal fields	lotId, tournamentId, eventType (bid | undo | sale | unsold), teamId, bidderId, amount, previousAmount, previousLeaderTeamId, roundNumber, outcome (accepted | rejected), rejectionReason, createdAt, reversesBidId, reason, performedBy.
Relationships	-> auctionLots, teams, users.
Indexes	Index: lotId+createdAt. U partial: (lotId, roundNumber, teamId) where eventType=bid and outcome=accepted (one accepted bid per captain per round). Index: tournamentId.
Constraints / notes	Never updated or deleted by application code (no update/delete routes; repository exposes insert and read only). Undo = new event referencing the reversed bid. Rejected bids may be logged with a size cap/TTL for abuse analysis.

fixtures

Aspect	Design
Purpose	Scheduled and generated matches.
Principal fields	tournamentId, stage (group | league | knockout), groupKey, round, matchNumber, slotA/slotB {teamId | sourceRef {type: winnerOf|loserOf|groupRank, fixtureId|groupKey, rank}}, scheduledAt, venue, status (scheduled | ongoing | completed | postponed | cancelled), resultId, advancesTo {fixtureId, slot}, manualOverride {by, at, reason}, scheduleHistory[] (bounded), version.
Relationships	-> tournaments, teams, matchResults.
Indexes	Index: tournamentId+stage+round+matchNumber (unique). Index: tournamentId+scheduledAt. Index: teams for participant queries.
Constraints / notes	Dependencies embedded as sourceRef because they are tiny, always read with the fixture, and need no independent lifecycle.

matchResults

Aspect	Design
Purpose	Current result state for a fixture (drafts and confirmed).
Principal fields	fixtureId, tournamentId, status (draft | previewed | confirmed), source (manual | csv), outcome {winnerTeamId, draw, scoreSummary}, stats {team[], player[]} (only supported fields), importJobId, confirmedBy, confirmedAt, currentRevision.
Relationships	-> fixtures, importJobs, resultRevisions.
Indexes	U: fixtureId. Index: tournamentId+status.
Constraints / notes	Public APIs return only status=confirmed data. Stats bounded by match size.

resultRevisions

Aspect	Design
Purpose	Immutable history of every confirmed result and correction.
Principal fields	resultId, fixtureId, revisionNumber, payload (full result and stats), reason, actedBy, actedAt, supersedes, impact {standingsDiffId, bracketImpact}.
Relationships	-> matchResults, standingsSnapshots.
Indexes	U: (resultId, revisionNumber).
Constraints / notes	Append-only. Revision 1 is the original confirmation.

importJobs

Aspect	Design
Purpose	CSV uploads awaiting organizer review.
Principal fields	fixtureId, uploadedBy, fileName, sizeBytes, sha256, adapterId/version, parsedRows (bounded), errors[], warnings[], status (preview | confirmed | discarded), createdAt, expiresAt.
Relationships	-> fixtures, users.
Indexes	Index: fixtureId+status. TTL on expiresAt for discarded/unconfirmed jobs.
Constraints / notes	Raw file not retained by default. Confirmed data is copied to matchResults. Schema depends on the real sample (Phase S).

standingsSnapshots

Aspect	Design
Purpose	Standings and qualification at a point in time.
Principal fields	tournamentId, stage, groupKey, computedAt, trigger (result_confirmed | correction | rule_change | override | finalization), tables[] (rows with supported fields), qualification[], overrides[], basedOnRevisionIds, isCurrent.
Relationships	-> tournaments, resultRevisions.
Indexes	Index: tournamentId+stage+groupKey+computedAt. U partial: tournamentId+stage+groupKey where isCurrent=true.
Constraints / notes	Snapshots are written in the same transaction as the triggering change. Size is bounded by team count.

announcements

Aspect	Design
Purpose	Tournament announcements.
Principal fields	tournamentId, authorId, title, body, pinned, status (draft | published | withdrawn), publishedAt, history[] (edits).
Relationships	-> tournaments, users.
Indexes	Index: tournamentId+pinned+publishedAt.
Constraints / notes	Only one pinned notice per tournament (enforced in service; optional partial unique index).

notifications

Aspect	Design
Purpose	Persistent per-user notifications.
Principal fields	recipientId, type, message, entityRef {type, id}, tournamentId, readAt, createdAt, dedupeKey.
Relationships	-> users, tournaments.
Indexes	Index: recipientId+readAt+createdAt. U partial: recipientId+dedupeKey. TTL or archive policy (e.g. 180 days) once decided.
Constraints / notes	Written inside the same transaction as the event where practical, otherwise by an outbox-style retry.

auditLogs

Aspect	Design
Purpose	Append-only audit trail.
Principal fields	actorId, actorRole, action, entityType, entityId, tournamentId, before, after, reason, context {ipHash, requestId}, createdAt, adminIntervention (bool).
Relationships	-> users, any entity (by type + id).
Indexes	Index: tournamentId+createdAt, actorId+createdAt, entityType+entityId+createdAt, action+createdAt.
Constraints / notes	No update/delete code path; DB user permissions can further restrict. Exceptional admin corrections are themselves audited with adminIntervention=true. Sensitive values (passwords, tokens) never logged.

counters

Aspect	Design
Purpose	Atomic sequence counters.
Principal fields	key (e.g. waitlist:), value.
Relationships	Logical only.
Indexes	U: key.
Constraints / notes	Incremented with findOneAndUpdate $inc. Also stores per-tournament capacity counters if used.

3. Embedded versus referenced data

Data	Decision	Reason
Tournament rules	Embedded in tournaments	Small, always read together; history kept in ruleRevisions to avoid unbounded growth
Registration settings	Embedded	Fixed small shape
Squad members	Referenced (teamAssignments)	Needs history and unique-active guarantee; squad can change over time
Bids	Separate collection	Unbounded growth, append-only audit, concurrency isolation from lot
Auction config/state	Own collection (auctionSessions)	Keeps tournaments clean; version-controlled; one write hotspot
Fixture dependencies (winnerOf, groupRank)	Embedded on fixture	Tiny, 1:1, read with fixture
Result stats	Embedded in matchResults and copied to resultRevisions	Bounded by match size; revisions must be self-contained and immutable
Schedule history	Embedded bounded array on fixture + audit log	Quick display; full detail in audit
Account request status history	Small embedded array	Few entries
Notifications	Separate collection	High volume, per-user queries
Standings table rows	Embedded in snapshot	One snapshot per stage/group; bounded by team count
Audit before/after values	Embedded in audit row (trimmed to changed fields)	Self-contained evidence

4. Entity relationship overview

```
users 1---* sessions
users 1---* organizerApplications        users 1---* accountRequests
users(organizer) 1---* tournaments
tournaments 1---* tournamentRuleRevisions   tournaments 1---* invites
tournaments 1---* registrations *---1 users(player)
tournaments 1---* teams 1---* teamAssignments *---1 users(player)
tournaments 1---1 auctionSessions 1---* auctionLots *---1 users(player)
auctionLots 1---* auctionBids *---1 teams
auctionLots 0..1---1 teamAssignments (method=auction)
tournaments 1---* fixtures *---* teams (via slots)
fixtures 1---0..1 matchResults 1---* resultRevisions
importJobs *---1 fixtures      importJobs 0..1---1 matchResults
tournaments 1---* standingsSnapshots
tournaments 1---* announcements   users 1---* notifications
auditLogs *---1 users(actor); auditLogs reference any entity
```

Reading it: a **tournament** owns everything tournament-scoped; **registrations** link users to tournaments; **teams + teamAssignments** convert registered players into squads; the **auction** trio (session, lots, bids) writes assignments when a player is sold; **fixtures** reference teams and each other; **results** hang off fixtures and drive **standingsSnapshots**; **notifications** and **auditLogs** observe all of it.

5. Integrity rules and how each is enforced

Rule (Blueprint)	Enforcement
No duplicate active registration	Partial unique index on registrations (tournamentId, playerId) for active statuses; service catches duplicate-key error and returns 409
Concurrent registration vs capacity	Transaction: conditional increment of capacity counter (filter value < max); on failure, place on waiting list with counter-based position
Only registered, eligible players assigned/auctioned	Service checks registration.status=approved and auctionEligible=true; lots are created only from this set
Player on one team only	Partial unique index on teamAssignments active (tournamentId, playerId) + unique lot per player per session
Sold lot has team and price	Service finalization transaction sets both; test and optional Mongo JSON-schema validator on auctionLots
Purse and squad limits	Conditional $inc on teams: filter purseRemaining >= price AND squadCount < squadMax, inside the sale transaction; also checked before accepting a bid
One bid per captain per round	Partial unique index on accepted bids (lotId, roundNumber, teamId)
Concurrent bids	Single conditional update on lot (status, version, currentPrice); losers get stale-bid rejection; optional per-lot in-process mutex
Unreviewed imports don't affect standings	Standings engine reads only matchResults.status=confirmed; importJobs are never read by standings
Corrections preserve originals	resultRevisions is insert-only; matchResults.currentRevision pointer moves forward
Results vs standings consistency	Confirm/correct/override/rule-change run in one transaction that writes result, revision, fixture advancement, snapshot and audit
Deletion keeps history	Anonymize users in place; no cascading deletes anywhere
Organizer ownership	Middleware loads tournament and compares organizerId (or admin); never trusts client-supplied ids
Administrator role	Partial unique index (only one admin) + server-side role checks + create-admin script only
Audit immutability	No mutation code path; consider separate DB role without update/delete on auditLogs and auctionBids

6. Transaction-sensitive workflows

All require a replica set (Atlas shared/dedicated tiers and the in-memory replica-set test helper provide this). Keep transactions short; no network calls inside them; retry on TransientTransactionError.

Workflow	Documents changed atomically
Register / promote from waiting list	registrations, counters, notification, audit
Approve organizer	organizerApplications, users, notification, audit
Accept bid	auctionLots, auctionBids (and auctionSessions.roundNumber)
Undo last bid	auctionLots, auctionBids (compensating event), auctionSessions (cancel countdown), audit
Sell lot (countdown expiry)	auctionLots, auctionBids(sale event), teams (purse, squadCount), teamAssignments, notification
Manual assignment / reassignment	teamAssignments (supersede + insert), teams counters, audit
Confirm result	matchResults, resultRevisions, fixtures (+ downstream slot), standingsSnapshots, notifications, audit
Correct result	same as confirm, plus impact record
Rule change with recalculation	tournaments, ruleRevisions, standingsSnapshots, affected fixtures, audit
Account deletion processing	users (anonymize), sessions (revoke), accountRequests, audit
Recovery completion	users.passwordHash, accountRequests, sessions (revoke all), audit
Auction start / finalize	auctionSessions, lots (create/validate), teams, tournaments.status, audit

7. Historical retention and audit requirements

- **Never delete:** auditLogs, auctionBids, resultRevisions, ruleRevisions, teamAssignments (superseded rows), registrations (status-based).
- **Expire:** sessions (TTL), importJobs unconfirmed (TTL ~7 days), rejected bid noise (optional cap), notifications (retention policy to be decided, suggested 180 days after read).
- **Anonymize on deletion:** users shell remains; public pages show "Deleted player"; audit retains ObjectId only.
- **Audit content:** actor, role, action, entity, timestamp, before/after (changed fields), reason (mandatory for rule edits, cancellations, reschedules, corrections, undo, removals, interventions).
- **Backups:** logical dumps on a schedule plus Atlas snapshots if available; restore drill before launch.

8. Recommended schema implementation order

Order	Models	Introduced in phase	Why this order
1	users, sessions	G	Everything depends on identity
2	auditLogs, counters	G / J	Needed by every later write
3	organizerApplications, accountRequests	G	Admin bootstrap and approval
4	tournaments, tournamentRuleRevisions	H / I	Root aggregate
5	invites, registrations	J	Participation
6	notifications (model early, wiring later)	U (stub in J)	Hooks emit from J onward
7	teams, teamAssignments	K	Squads, manual teams
8	auctionSessions, auctionLots, auctionBids	L / M	Auction engine
9	fixtures	N	Brackets and schedules
10	matchResults, resultRevisions, standingsSnapshots	P / R	Standings and results
11	importJobs	S	After CSV sample arrives
12	announcements	U	Communication

9. How the schema supports the main workflows

Workflow	Data path
Public browsing	tournaments (+ teams, fixtures, standingsSnapshots current, announcements) read-only; confirmed results only
Registration	registrations + counters (transaction); waitlist order via counter; invites for invite-only
Captain and eligibility	registrations.captainInterest / auctionEligible; teams.captainId
Auction	auctionSessions -> lots -> bids; sale writes teamAssignments and updates teams counters atomically
Fixtures and brackets	fixtures with embedded slot sources; confirmation fills downstream slots
Results and corrections	matchResults + resultRevisions; confirmation triggers snapshot recalculation
Notifications and announcements	notifications per recipient; announcements per tournament; sockets only push
Privacy and deletion	users.settings.historyVisibility filtering in read services; anonymization keeps references valid
Oversight	auditLogs queried by tournament, actor, entity